Part 1: Data Structures in R

Summary

This Part begins our exploration of basic concepts of the R programming language.

The development here follows closely that of the textbook *Advanced R* by Hadley Wickham, CRC Press, 2014.

It is assumed that you are somewhat familiar with basic programming concepts, and that you have installed R and RStudio on your computer.

Getting Help

The most important function in R is `help()`. For any function described below, there are almost always additional arguments that can be specified to extend the usefulness of the function.

For example, looking at `help(mean)` tells the user how to adjust the result to calculate the trimmed mean, and how to have the function handle missing values.

See part of the output on the following slide.

In addition, the webpage http://Rseek.org is a useful Google-like resource for finding information regarding R.

Description

Generic function for the (trimmed) arithmetic mean.

Usage

```
mean(x, ...)

## Default S3 method:
mean(x, trim = 0, na.rm = FALSE, ...)
```

Arguments

`x`	An `R` object. Currently there are methods for numeric/logical vectors and date, date-time and time interval objects. Complex vectors are allowed for `trim = 0`, only.
`trim`	the fraction (0 to 0.5) of observations to be trimmed from each end of `x` before the mean is computed. Values of trim outside that range are taken as the nearest endpoint.
`na.rm`	a logical value indicating whether `NA` values should be stripped before the computation proceeds.
`...`	further arguments passed to or from other methods.

Figure 1: A portion of the output of `help(mean)`.

Objects in R

R is an object oriented language. During a given R session, one has any number of objects in the current environment. These objects will include vectors, matrices, lists, data frames, user-defined functions, and more.

The function `str()` provides useful information regarding any object.

Use `objects()` to see the current list of objects. The function `rm()` will remove an object.

When exiting R using the `q()` function, the user is given the opportunity to save all current objects; these will be reloaded upon restarting R in that directory. See the functions `save()` and `load()` for storing selected R objects.

Exercise: Describe a situation in which the `save()` function would be useful. Can a single call to `save()` be used with multiple objects?

Vectors

Vectors are the fundamental object in R and come in four basic types:

1. logical
2. integer
3. double
4. character

The function `c()` creates a vector.

The function `typeof()` will return the type of any vector.

There are logical test functions `is.logical()`, `is.integer()`, `is.double()`, and `is.character()`.

Consider the following example:

```
> logex = c(TRUE,TRUE,FALSE) # These two lines define
> logex = c(T,T,F)           # the same logical vector
> typeof(logex)
[1] "logical"
```

```
> is.logical(logex)
[1] TRUE
```

```
> is.integer(logex)
[1] FALSE
```

```
> length(logex)
[1] 3
```


Integer vectors require the use of the `L` suffix:

```
> intex = c(4L, 2L, 6L, 1L)
> typeof(intex)
[1] "integer"
```

```
> is.double(intex)
[1] FALSE
```

```
> doubex = c(4,2,6,1)
> typeof(doubex)
[1] "double"
```

```
> is.integer(doubex)
[1] FALSE
```

Consider a simple example of a character vector:

```
> charex = c("Hello", "World")
```

and note the behavior of `is.numeric()`:

```
> is.numeric(logex)
[1] FALSE
```

```
> is.numeric(intex)
[1] TRUE
```

```
> is.numeric(doubex)
[1] TRUE
```

```
> is.numeric(charex)
[1] FALSE
```


Coercion

Vectors can only have elements of a single type. Consider the behavior of the following example:

```
> mixex = c(1, 2, "Q")
> print(mixex)
[1] "1" "2" "Q"
```

```
> typeof(mixex)
[1] "character"
```

The vector `mixex` is "coerced" into type character in a natural way.

There are functions `as.logical()`, `as.integer()`, `as.double()`, and `as.character()` that will force coercion when possible.

`TRUE` and `FALSE` will be coerced into `1` and `0`, respectively, and vice-versa.

```
> ex1 = c(1,0,T)
> print(ex1)
[1] 1 0 1
```

```
> typeof(ex1)
[1] "double"
```

```
> as.logical(c(1,1,0,1))
[1]  TRUE  TRUE FALSE  TRUE
```

Some functions will automatically coerce the vector:

```
> sum(c(T,T,T,F))
[1] 3
```


Exercise: Inspect `help(rnorm)`. Use this to approximate the probability that a standard normal random variable exceeds 1.0.

Missing Values in R

The symbol `NA` is reserved by R to denote "missing" values (interpreted broadly). Including an `NA` in a vector will not coerce the vector into a character type. (Do not wrap the `NA` in quotes.)

```
> is.numeric(c(45, 78, 12, NA))
[1] TRUE
```

```
> is.na(c(45, 78, 12, NA))
[1] FALSE FALSE FALSE  TRUE
```

Want to test for missing values? The `any()` function is helpful:

```
> any(is.na(c(45, 78, 12, NA)))
[1] TRUE
```


Creating Vectors

There are alternatives to using `c()`:

```
> 1:10
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
> seq(1,10,by=2)
[1] 1 3 5 7 9
```

```
> seq(1,20,length=5)
[1]  1.00  5.75 10.50 15.25 20.00
```

There are functions `logical()`, `integer()`, `double()`, and `character()` which initialize vectors of the specified length.

Subsetting Vectors

Single square brackets are used to extract entries of a vector:

```
> vectex = seq(1,20,by=3)
> vectex[3]
[1] 7
```

```
> vectex[3:6]
[1]  7 10 13 16
```

```
> vectex[c(4,7,2)]    # You can reorder entries
[1] 10 19  4
```

```
> vectex[-c(4,7,2)]   # Exclude entries 4, 7, 2
[1]  1  7 13 16
```


Exercise: Run the following code, and describe what happened:

```
> x = rnorm(5)
> x[order(x)]
```

Lists

Lists in R can consist of elements of various types, including vectors of different lengths:

```
> listex = list(c(T,F,T), 1:5, 6)
> print(listex)
[[1]]
[1]  TRUE FALSE  TRUE

[[2]]
[1] 1 2 3 4 5

[[3]]
[1] 6
```


Using a single square bracket subsets a list into a new list, while using double brackets pulls out a component of a list:

```
> listex[2]     # This is a list with a single component
[[1]]
[1] 1 2 3 4 5
```

```
> typeof(listex[2])
[1] "list"
```

```
> listex[[2]]   # This is an integer vector of length 5
[1] 1 2 3 4 5
```

```
> is.list(listex[[2]])
[1] FALSE
```

Exercise: Consider the R command `unlist(listex[2:3])`. Describe the behavior of this function. What about `unlist(listex)`?

Components of a list can be named, then referenced using the `$` operator:

```
> names(listex) = c("AA", "BB", "CC")
> names(listex)
[1] "AA" "BB" "CC"
```

```
> listex$AA
[1]  TRUE FALSE  TRUE
```

```
> listex$CC
[1] 6
```

The output of many of the more sophisticated statistical functions in R is a named list of results from the procedure.

Exercise: Execute the following commands:

```
> x = rnorm(100)
> y = rnorm(100)
> holdlmout = lm(y~x)
```

What happened here? Inspect the form of `holdlmout`.

Attributes

The names seen above are a special case of an attribute of an R object. Any R object can be given arbitrary attributes using the `attr()` function:

```
> x = seq(20, 1, by=-2)
> attr(x, "keyproperty") = "A reversed sequence"
> attr(x, "keyproperty")
[1] "A reversed sequence"
```

```
> attributes(x)
$keyproperty
[1] "A reversed sequence"
```

Factors

Categorial variables are called factors in R. The `factor()` function defines a variable as a factor, and gives it an attribute consisting of the levels of that factor, i.e., the possible values:

```
> finalgrade = factor(c("C","B","B","C","A"))
> levels(finalgrade)
[1] "A" "B" "C"
```

```
> finalgrade = factor(c("C","B","B","C","A"),
+                     labels=c("C","B","A"), ordered=T)
> levels(finalgrade)
[1] "C" "B" "A"
```


Exercise: How does R internally represent a factor? Consider the following example to see why you should be careful:

```
> as.integer(factor(c(0,1,2)))
[1] 1 2 3
```

Matrices

A matrix in R is two-dimensional array of elements of the same type. The function `matrix()` creates a matrix from a vector, constructing it column-by-column (unless you set `byrow=TRUE`):

```
> matex = matrix(1:6, nrow=2)
> print(matex)
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```
> c(nrow(matex),ncol(matex))
[1] 2 3
```


The functions `cbind()` and `rbind()` are often used to build matrices from vectors and/or other matrices:

```
> rbind(1:3,4:6,7:9)
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

```
> cbind(cbind(1:3,4:6,7:9), rbind(1:3,4:6,7:9))
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    1    4    7    1    2    3
[2,]    2    5    8    4    5    6
[3,]    3    6    9    7    8    9
```

Exercise: Suppose that `n` and `m` are defined, and each is a positive integer. Write a single line of R code that creates a `n` by `m` matrix whose i^{th} row is filled with i. Look at `help(rep)`.

Arrays

Arrays are generalizations of matrices, and can be of any dimension. This is a three-dimensional array:

```
> arrex = array(1:20, dim=c(2,5,2))
> dim(arrex)
[1] 2 5 2
```

```
> is.array(arrex)
[1] TRUE
```

```
> is.matrix(arrex)
[1] FALSE
```

```
> print(arrex)
, , 1

     [,1] [,2] [,3] [,4] [,5]
[1,]    1    3    5    7    9
[2,]    2    4    6    8   10

, , 2

     [,1] [,2] [,3] [,4] [,5]
[1,]   11   13   15   17   19
[2,]   12   14   16   18   20
```


Subsetting Matrices and Arrays

Square brackets are also used to subset matrices and arrays:

```
> matex[1,2]
[1] 3
```

```
> arrex[2,1,2]
[1] 12
```

```
> matex[,1:2]
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

R will simplify an object when it feels it's appropriate...

```
> matex[1,1:2]
[1] 1 3
```

```
> is.matrix(matex[1,1:2])
[1] FALSE
```

...unless the `drop = FALSE` argument is provided:

```
> matex[1,1:2, drop=F]
     [,1] [,2]
[1,]    1    3
```

```
> is.matrix(matex[1,1:2, drop=F])
[1] TRUE
```


Exercise: Consider the following code:

```
> x = list("stats", 1:5, T, -10)
> dim(x) = c(2,2)
```

Describe the contents of `x`.

Data Frames

Data frames in R are generalizations of matrices in that the columns of a data frame can be of different types. They are the primary way in which data sets are stored, given that most data we obtain is a mixture of numerical information, text, and categorical variables (factors).

The function `data.frame()` creates the data frame:

```
> gradebook = data.frame(
+     Name = c("Miguel","Yotam","Mary","Jing"),
+     HWScore = c(98, 87, 99, 78),
+     ExamScore = c(92, 91, 87, 95),
+     FinalGrade = c("A", "A-", "A", "B"),
+     stringsAsFactors = FALSE)
```


Exercise: What happens if the argument `stringsAsFactors` is not set to `FALSE`? How could one treat `Name` as text data, and `FinalGrade` as a factor?

R stores a data frame as a list of vectors. Like other lists, components can be referenced using the `$` operator, and components can be easily added:

```
> mean(gradebook$HWScore)
[1] 90.5
```

```
> gradebook$SID = c("3782", "2390", "4623", "0376")
> print(gradebook)
    Name HWScore ExamScore FinalGrade  SID
1 Miguel      98        92          A 3782
2  Yotam      87        91         A- 2390
3   Mary      99        87          A 4623
4   Jing      78        95          B 0376
```


Logical Subsetting

An alternative to subsetting with a vector of positions is to use a logical vector, with `TRUE` in the entries to be retained, and `FALSE` in those positions to be excluded:

```
> x = c(45,12,78,32,89)
> x[c(T,T,F,F,T)]
[1] 45 12 89
```

A link is provided by the `which()` function. This handy function simply takes a logical vector and returns an integer vector consisting of the positions of its `TRUE` entries:

```
> which(c(T,T,F,F,T))
[1] 1 2 5
```

Exercise: What happens when the length of the subsetting logical vector does not match the length of the original vector?

Use conditional statements to extract a portion of a data frame. The operator == indicates "equals" while != is "not equals to."

```
> gradebook[gradebook$ExamScore > 90,]
    Name HWScore ExamScore FinalGrade  SID
1 Miguel      98        92          A 3782
2  Yotam      87        91         A- 2390
4   Jing      78        95          B 0376
```

```
> gradebook[gradebook$FinalGrade == "A",]
    Name HWScore ExamScore FinalGrade  SID
1 Miguel      98        92          A 3782
3   Mary      99        87          A 4623
```

Part 2: Tour of R Functions

Summary

In this Part we take a whirlwind tour through some of the most used functions in R.

Statistical analysis procedures are omitted at this point. Also, functions generally used in writing code (e.g., `while()`, etc.), along with graphics/visualization functions, are left to other Parts.

Instead, focus is placed on functions for manipulating vectors and matrices and performing crucial mathematical calculations.

Getting Going, and Quitting

After starting R, ensure you are in the desired working directory. R will write and read files, and otherwise store data in this directory. The working directory is set using the `setwd()` function:

```
> setwd("/Users/chadschafer")
> getwd()
[1] "/Users/chadschafer"
```

To quit R, use `q()`:

```
> q()
```

You will be asked if you want to save the current objects. If you do so, R will create a (hidden) file in the current working directory that will automatically be reloaded when you return to this working directory.

Argument Matching

As seen previously, the `help()` function provides detailed argument lists for each function. The `args()` function can also be used:

```
> args(sample)
function (x, size, replace = FALSE, prob = NULL)
NULL
```

Here we see that there are four arguments to `sample()`. Two of them have default values: `replace = FALSE` and `prob = NULL`.

When calling `sample()`, one can either provide arguments with names, in any order, or without names, in the order as they are given in the `args(sample)` output.

Also, only enough of the argument name need be provided that uniquely specifies the argument.

In other words, these commands will all work equally well:

```
> sample(1:100, 10, FALSE)
> sample(1:100, size=10, replace=FALSE)
> sample(x=1:100, rep=F, si=10)
> sample(1:100, 10, r=F)
```

Basic Math Functions

Each of the following does exactly what you would expect when passed a numeric (integer or double) vector `x`. As usual, see `help()` for arguments that expand/alter their capabilities.

```
> sum(x)
> max(x)
> min(x)
> sort(x)
> rank(x)
> mean(x)
> var(x)
> sd(x)
```


Handling Missing Values

Most of the functions on the previous page have arguments that control how `NA` values should be handled. Often, but not always, this argument is named `na.rm` (to stand for "NA Remove") and by default it is set to `FALSE`, meaning that `NA` values in the vector will result in `NA` output.

```
> max(c(45, 78, 12, NA))
[1] NA
```

```
> max(c(45, 78, 12, NA), na.rm = TRUE)
[1] 78
```

`sort()` and `rank()` are special cases. By default, `NA` values are removed, but see argument `na.last`.

Mathematical functions that take scalar inputs will return a vector when passed a vector:

```
> sqrt(c(1,2,3))
[1] 1.000000 1.414214 1.732051
```

```
> log(x) # Default is natural log
> exp(x)
> abs(x)
> tan(x)
> round(x)
> floor(x)
> ceiling(x)
> factorial(x)
```


Vector Arithmetic

Vectors and matrices of the same size can be added and subtracted, resulting in another object of the same size.

```
> c(4,5,6) - c(3,2,1)
[1] 1 3 5
```

Multiplication using $*$ is element-by-element, **even with matrices**:

```
> matrix(c(1,2,3,4),nrow=2) * matrix(c(1,2,3,4),nrow=2)
     [,1] [,2]
[1,]    1    9
[2,]    4   16
```

The operator %% finds the "modulus."

Exercise: What happens when one tries to add/subtract/multiply vectors of differing lengths?

Logical Operators

The operators `|` and `&` perform elementwise logical OR and logical AND comparisons, respectively:

```
> c(T,T,F) | c(T,F,F)
[1]  TRUE  TRUE FALSE
```

```
> c(T,T,F) & c(T,F,F)
[1]  TRUE FALSE FALSE
```

The `!` operator negates:

```
> !(c(T,T,F) | c(T,F,F))
[1] FALSE FALSE  TRUE
```

The functions `any()`, `all()`, and `which()` take logical vectors as input:

Are **any** of the simulated values greater than 4?

```
> any(rnorm(1000)>4)
[1] FALSE
```

Are **all** of the simulated values less than 3?

```
> all(rnorm(1000)<3)
[1] FALSE
```

Which entries have values larger than 2.5?

```
> which(rnorm(1000)>2.5)
[1]  73 185 197 281 541 791 888
```


Set Functions

There are functions that treat vectors like sets:

```
> intersect(c(1,2,3),c(2,3,4,5))
[1] 2 3
```

```
> union(c(1,2,3),c(2,3,4,5))
[1] 1 2 3 4 5
```

```
> setdiff(c(1,2,3),c(2,3,4,5))
[1] 1
```

```
> setequal(c(1,2,3),c(3,2,1))
[1] TRUE
```

Exercise: Note that `setdiff()` is not symmetric. Write a line of R code that creates a symmetric difference between sets `x` and `y`.

Finding an Element

Operator `%in%` tests if a value is in a vector or matrix, and `match()` returns the **first** position of that value in a vector.

```
> "GOOG" %in% c("GOOG", "SBUX", "AAPL")
[1] TRUE
```

```
> match("SBUX", c("GOOG", "SBUX", "AAPL"))
[1] 2
```

```
> match(4,c(4,1,7,4,2,4))
[1] 1
```

```
> which(c(4,1,7,4,2,4) == 4)
[1] 1 4 6
```

Matrix Algebra

R has great capacity to do matrix algebra, albeit not at the same speed as Matlab. The operator to perform matrix multiplication is `%*%`. Other useful functions (assuming `A` is a matrix of form appropriate for the function):

```
> t(A)      #  Matrix Transpose
> nrow(A)
> ncol(A)
> chol(A)
> eigen(A)
> svd(A)
> qr(A)
> diag(A)   # Extract the diagonal
> solve(A)  # Matrix Inverse
```


Exercise: Look at `help(diag)` and explore the various ways this function gets utilized.

Working with Named Distributions

Each standard, “named" distribution has a suite of four useful functions for doing basic probability calculations.

For example, for the normal distribution:

Generate a (pseudo-)random sample of size 5 from the normal distribution with mean 20 and standard deviation 3:

```
> set.seed(0)
> rnorm(5, 20, 3)
[1] 23.78886 19.02130 23.98940 23.81729 21.24392
```

The function `set.seed()` controls the seed of the random number generator in R.

Evaluate the CDF: If X has the normal distribution with mean 10 and standard deviation 2, what is $P(X \leq 10.5)$?

```
> pnorm(10.5, 10, 2)
[1] 0.5987063
```

Find Quantiles: If Y has the normal distribution with mean 5 and standard deviation 3, what value of c is such that $P(Y \leq c) = 0.95$?

```
> qnorm(0.95, 5, 3)
[1] 9.934561
```

Evaluate the PDF: Evaluate the standard normal density at 0.5.

```
> dnorm(0.5) #  Defaults are mean=0, sd=1
[1] 0.3520653
```

Such functions exist for all of the common distributions:

Exponential: `rexp(),pexp(),qexp(),dexp()`

Gamma: `rgamma(),pgamma(),qgamma(),dgamma()`

Poisson: `rpois(),ppois(),qpois(),dpois()`

Binomial: `rbinom(),pbinom(),qbinom(),dbinom()`

Chi-Square: `rchisq(),pchisq(),qchisq(),dchisq()`

t: `rt(),pt(),qt(),dt()`

Uniform: `runif(),punif(),qunif(),dunif()`

Carefully inspect the arguments: There are different ways of parameterizing a distribution.

Exercise: Write code, without using `rnorm()`, to generate random variables from the normal distribution.

Assigning Values to Objects

In some situations it can be very useful to dynamically construct and reference object names. The `assign()` and `get()` functions enable this.

```
> objname = "x"
> assign(objname, 5)
> print(x)
[1] 5
```

```
> get(objname)
[1] 5
```


The `do.call()` function extends this idea to calling functions. Here, the function to be called is provided first, followed by the arguments to that function as a list:

```
> do.call("mean", list(1:10, trim = 0.1))
[1] 5.5
```

Sampling a Vector

The `sample()` function gets a great deal of use, e.g. when generating a random subsample of a data set, when running a bootstrap, when randomly permuting a list, etc.

One specifies the vector from which to sample, the size of the sample to draw, whether sampling is with or without replacement:

```
> sample(1:10, 5, FALSE)
[1]  9  3 10  5  6
```

```
> sample(1:10, 5, TRUE)
[1]  3  9 10  7  7
```


Exercise: In K-Fold Cross Validation a data set is divided into `K` distinct groups, of size as close to equal as possible. Analysis then proceeds with one fold omitted in order to test performance. Let `n` denote the sample size. Construct an object `fold` which holds the fold assignment, determined at random.

Apply Functions

R has a family of apply functions that allow the user to efficiently run another function on subsets of another object. The simplest example is the use of `apply()` on a matrix. Here, `matex` is a 2 by 5 matrix:

```
> matex = matrix(1:10, nrow=2)
```

Find the sum of each row, and the mean of each column:

```
> apply(matex,1,sum)
[1] 25 30
```

```
> apply(matex,2,mean)
[1] 1.5 3.5 5.5 7.5 9.5
```

This function can also be used on an array.

The `sweep()` function is often used in combination with `apply()`. It allows the user to perform an operation on every row or column of a matrix. The result is a matrx of the same dimension of the original matrix.

The following example "centers" each column of `matex` on zero:

```
> sweep(matex, MAR = 2, apply(matex,2,mean), FUN="-")
     [,1] [,2] [,3] [,4] [,5]
[1,] -0.5 -0.5 -0.5 -0.5 -0.5
[2,]  0.5  0.5  0.5  0.5  0.5
```

Here, `FUN` specifies the operation to be applied on each column (since `MAR = 2`). The amount to be subtracted is each column mean, as calculated by `apply(matex,2,mean)`.

There are additional apply functions to be used on more complex objects. For example, the function `lapply()` works on the components of a list:

```
> listex = list(c(T,F,T), 1:5, c("a","b","c","d"))
> lapply(listex, typeof)
[[1]]
[1] "logical"

[[2]]
[1] "integer"

[[3]]
[1] "character"
```


Exercise: Take a look at `help(sapply)`. How does this function differ from `lapply()`?

Split

The `split()` function breaks an object into a list using a specified grouping variable. First, a simple example:

```
> split(1:5, c(1,1,2,2,2))
$`1`
[1] 1 2

$`2`
[1] 3 4 5
```


In our "gradebook" example, `split()` can group the rows:

```
> split(gradebook, gradebook$FinalGrade)
$A
    Name HWScore ExamScore FinalGrade
1 Miguel      98        92          A
3   Mary      99        87          A

$`A-`
   Name HWScore ExamScore FinalGrade
2 Yotam      87        91         A-

$B
  Name HWScore ExamScore FinalGrade
4 Jing      78        95          B
```

Exercise: How could I find the average homework score for students of each final letter grade?

Working with Strings

R has an range of functions for working with strings. The most commonly used includes `paste()`, for concatenating strings. Coercion of non-string types is done automatically.

```
> paste("abc","def")
[1] "abc def"
```

```
> paste("abc","def",sep="-")
[1] "abc-def"
```

```
> paste0("abc",30)
[1] "abc30"
```

Note that `paste0` uses `sep=""` by default.

Substrings can be extracted or assigned using `substr()`:

```
> strex = "abcdef"
> substr(strex,2,4)
[1] "bcd"
```

```
> substr(strex,3,5) = "qed"
> print(strex)
[1] "abqedf"
```

Note that this syntax does **not** find the substring:

```
> strex[2:4]
[1] NA NA NA
```


Exercise: Explain the behavior seen in the previous example. Also, look at `length(strex)` and `nchar(strex)` for a related issue.

The function `strsplit()` breaks a string into components based on a provided "split" string:

```
> strsplit("file_00001_A", "_")
[[1]]
[1] "file"  "00001" "A"
```

A vector of strings becomes a list:

```
> strsplit(c("file_00001_A", "file_00002_A"), "_")
[[1]]
[1] "file"  "00001" "A"

[[2]]
[1] "file"  "00002" "A"
```


Exercise: Suppose that object `fnamelist` is a vector consisting of filenames, each is of the form `A_B_C`, where `B` is a number with leading zeros to force it to be of length 5. Your objective is to extract the numbers from this center section, in numeric form. How would you do this (1) if the `A` portion is always the same length, (2) if the length of the `A` portion varies from file to file?

Installing Packages

The development of R is largely a decentralized effort, with users working to extend its functionality with add-on packages built following a stanard format to enable others to easily install and utilize these new functions.

The CRAN website https://cran.r-project.org/web/packages/ has a list of packages that have been posted to their repository.

The range of packages available is staggering, with almost any functionality you could imagine have been implemented by someone. Quality can vary, however, and there can be inconsistency in how R syntax is utilized.

Packages can be installed from the command line. For example, package `tidyverse` is very useful, enabling a wide range of advanced data manipulation and visualization tools that we will use later on

```
> install.packages("tidyverse")
```

A package only needs to be installed once, but it needs to loaded into each R session where it is to be used. This is achieved with the `library()` function:

```
> library(tidyverse)
```

Package `dplyr`

Some of the capabilities of package `dplyr`, which is installed as part of `tidyverse`, are worth pointing out now.

The `filter()` function allows one to easily extract particular rows of a data frame based on one or multiple conditions:

```
> filter(gradebook, HWScore > 90 & ExamScore > 90)
    Name HWScore ExamScore FinalGrade
1 Miguel      98        92          A
```

```
> filter(gradebook, ExamScore >= 95 | FinalGrade == "A")
    Name HWScore ExamScore FinalGrade
1 Miguel      98        92          A
2   Mary      99        87          A
3   Jing      78        95          B
```


The `arrange()` function will reorder the rows of a data frame using the specified column(s). The `desc()` function switches to descending order.

```
> arrange(gradebook,HWScore)
    Name HWScore ExamScore FinalGrade
1   Jing      78        95          B
2  Yotam      87        91         A-
3 Miguel      98        92          A
4   Mary      99        87          A
```

```
> arrange(gradebook,desc(HWScore))
    Name HWScore ExamScore FinalGrade
1   Mary      99        87          A
2 Miguel      98        92          A
3  Yotam      87        91         A-
4   Jing      78        95          B
```

Multiple columns can be provided to break ties.

The `select()` function extracts particular variables from a data frame.

```
> select(gradebook, Name, FinalGrade)
Warning: `lang()` is deprecated as of rlang 0.2.0.
Please use `call2()` instead.
This warning is displayed once per session.
Warning: `new_overscope()` is deprecated as of rlang 0.2.0.
Please use `new_data_mask()` instead.
This warning is displayed once per session.
Warning: `overscope_eval_next()` is deprecated as of rlang 0.2.0.
Please use `eval_tidy()` with a data mask instead.
This warning is displayed once per session.
    Name FinalGrade
1 Miguel          A
2  Yotam         A-
3   Mary          A
4   Jing          B
```


Look at `help(select_helpers)` for functions that can make things even easier, e.g.,

```
> select(gradebook, ends_with("Score"))
  HWScore ExamScore
1      98        92
2      87        91
3      99        87
4      78        95
```

Part 3: Writing R Functions

Summary

This Part covers the programming constructs needed to write R functions.

A Simple Example

First, we start with a simple example of an R function:

```
> whichclosest = function(z, na.rm=FALSE)
+ {
+   if((!na.rm & any(is.na(z))) | sum(!is.na(z)) <= 1)
+   {
+     return(NA)
+   }
+
+   zsrt = sort(z, index.return=TRUE, na.last=TRUE)
+   minpos = which.min(diff(zsrt$x))
+   return(c(zsrt$ix[minpos], zsrt$ix[minpos+1]))
+ }
```

First, note how the arguments to the function `whichclosest()` are specified, and that they can be given default values. (In the example, `na.rm` has value `FALSE` by default.)

Then, note how curly brackets `{}` are used to wrap the body of the function. Curly brackets are also used to wrap chunks of code, e.g., following an `if()` statement.

Finally, the `return()` function exits and passes back output. The call to `return()` at the very end of a function is implied and can be omitted, i.e., the final line could have just been:

```
c(zsrt$ix[minpos], zsrt$ix[minpos+1])
```


Exercise: Make sure to understand the functions and their usage in `whichclosest()`.

Arguments

How function arguments are handled was discussed some in the preceding Part. Recall that arguments do not need to be provided in a fixed order if they are named by the calling procedure.

Also, recall the useful `args()` function; it works equally well on user-defined functions.

The default arguments to a function can depend on other arguments. In this example, the default behavior of `meanT()` is to return the 10% trimmed mean if the length of the data vector exceeds 500.

```
> meanT = function(x, trim = (length(x) > 500)*0.1)
+ {
+   return(mean(x, trim=trim))
+ }
>
> x = rnorm(600)
> meanT(x)
[1] -0.03314024
```

```
> meanT(x, trim=0)
[1] -0.01451877
```


Another useful feature when working with function arguments is the `...` syntax. If `...` is included at the end of an argument list, additional arguments can be passed into the function, and they will be passed on to other functions called within that function.

Typically this is utilized when the user-defined function calls a sophisticated function which can potentially have many arguments:

```
> newfunc = function(x,y,...)
+ {
+   #  Some R code here
+   holdlm = lm(y~x,...)
+   #  Some more R code here
+ }
```

Lexical Scoping

The term lexical scoping refers to general rules that R uses to find the values associated with objects.

The entire topic can be quite complicated, as R is capable of handling these situations in a sophisticated manner using a range of functions to create and alter the current environment.

At this stage, it is important to understand how R handles objects referred to within functions. We will do this through a series of simple examples.

R looks first within the function to see if an object is defined. If it is not, then it looks into the parent environment to see if it has a value.

In the example below, `x` is defined to have a value 10 outside of the function `funcex1()`, and this function does not itself define `x`. Hence, `funcex1()` uses `x` equal to 10.

```
> funcex1 = function(y)
+ {
+   return(x+y)
+ }
>
> x = 10
> funcex1(5)
[1] 15
```

If the function internally assigns a value to `x`, the function will use that. But, note that the value of `x` in the parent environment is unchanged.

```
> funcex2 = function(y)
+ {
+   x = 12
+   return(x+y)
+ }
>
> x = 10
> funcex2(5)
[1] 17
```

```
> print(x)
[1] 10
```


This illustrates that you can overwrite **some** objects (including functions) in the local environment that exist in a parent (including the system level):

```
> T = 0
> T == TRUE
[1] FALSE
```

```
> T == FALSE
[1] TRUE
```

After removing the newly-created object, R will again look into the parent frame for its value:

```
> rm(T)
> T == TRUE
[1] TRUE
```

Syntax such as the following will not alter the value of `x` in the parent environment:

```
> funcex3 = function(y)
+ {
+   x = x + 2
+   return(x+y)
+ }
>
> x = 10
> funcex3(5)
[1] 17
```

```
> print(x)
[1] 10
```


The `assign()` function does allow for altering objects in the parent environment. Use with `pos = 1`.

```
> funcex4 = function(y)
+ {
+   assign("x", x+2, pos=1)
+   return(x+y)
+ }
>
> x = 10
> funcex4(5)
[1] 17
```

```
> print(x)
[1] 12
```

Exercise: Look at `help(assign)` to understand how the `pos` argument works. Look also at `search()`.

The function `do.call()` can also be used in the global environment:

```
> funcex5 = function(trm=0)
+ {
+   xmean = do.call("mean", list(c(x), trim=trm),
+                   env=globalenv())
+   return(xmean)
+ }
>
> x = c(10:20, 1000)
> funcex5(0.1)
[1] 15.5
```

Things start to get odd when you have an argument to a function whose name matches that of an object in the parent frame:

```
> funcex6 = function(x,y)
+ {
+   return(x+y)
+ }
>
> x = 10
> funcex6(x=x, 5)
[1] 15
```

```
> funcex6(x=4, y=x)
[1] 14
```


Control Statements

The flow of the code can be controlled using standard constructs such as the `if-else` statement:

```
> funcex7 = function(x)
+ {
+   if(x > 0)
+   {
+     print("x is positive")
+   } else
+   {
+     print("x is not positive")
+   }
+ }
```

The `ifelse()` function gives a compact way of representing the conditional statement. It allows for vector conditions in a simple way.

The following example changes all `-999` in `x` into `NA`:

```
> x = ifelse(x == -999, NA, x)
```


The `for()` statement is most often used to loop over integers:

```
> holdlist = list(1:10, 5:20, 40:90)
> for(i in 1:length(holdlist))
+ {
+   print(mean(holdlist[[i]]))
+ }
```

but it can loop over any vector or list:

```
> for(x in holdlist)
+ {
+   print(length(x))
+ }
```

The `break()` and `next()` functions exit the `for()` loop and skip to the next iteration, respectively.

Exericise: Often `for()` loops can be removed and cleaner code results. Rewrite the code segments above without a `for()` loop.

It is a myth that `for()` loops are necessarily slower than other approaches in R. Consider the following example:

```
> x = matrix(rnorm(10000000),ncol=100000)
> system.time(apply(x,2,mean))
   user  system elapsed
  0.456   0.040   0.495
```

```
> meanout = numeric(ncol(x))
> system.time(for(i in 1:ncol(x))
+ {
+   meanout[i] = mean(x[,i])
+ })
   user  system elapsed
  0.336   0.027   0.362
```

You definitely should use vector arithmetic when possible, however:

```
> x = rnorm(10000000); y = rnorm(10000000)
> z = numeric(10000000)
>
> system.time(for(i in 1:length(z))
+ {
+   z[i] = x[i] + y[i]
+ })
   user  system elapsed
  0.701   0.020   0.721
```

```
> system.time(assign("z",x+y))
   user  system elapsed
  0.017   0.000   0.016
```


Exercise: Suppose that `meanvec` is a numeric vector. The goal is create a vector of psuedo-random variates drawn from the normal distribution with the mean of each draw given by the corresponding entry in `meanvec`. The variance is always one. Compare the time it takes to do this with and without a `for()` loop.

The `while()` function continues a loop until the given condition is false.

This example waits for a random sample with mean of at least 0.5:

```
> x = rnorm(100); nsamp = 1
> while(mean(x)<0.5)
+ {
+   x = rnorm(100)
+   nsamp = nsamp + 1
+ }
> mean(x)
[1] 0.527035
```

```
> print(nsamp)
[1] 174920
```


Returning Information

As seen above, the `return()` call exits the function. There can be multiple calls to `return()` in any one function.

The call

```
> return()
```

can be used to exit a function only returning the special symbol `NULL`. This can be useful if, for example, the function is only intended to generate a plot or file.

It is very common to see `return()` used along with `list()` to enable the function to return different types of information.

The `stop()` function can be used to exit a function with an error condition. It is a good idea to do this instead of using `return()` since R has special functions for handling errors. See also `warning()` for passing back warnings without exiting the function.

```
> funcex7 = function(x)
+ {
+   if(!is.numeric(x))
+   {
+     stop("Input argument must be numeric")
+   }
+   return(x^2)
+ }
```


Infix Operators

R also allows you to create infix operators. Consider the following example:

```
> "%_%" = function(str1,str2)
+ {
+   paste(str1,str2,sep="_")
+ }
> "filename" %_% 1 %_% 12
[1] "filename_1_12"
```

Note that the operator must start and end with `%`.

Exercise: Create a new addition operator `%+%` that returns `x` when given `x %+% NA`. (Note that the default addition operator would return `NA` in that case.) The new operator should still return `NA` when passed `NA %+% NA`.

Part 4: Understanding Data Sets

Summary

This Part presents an example of reading in a data set, and ensuring the data is fully understood and properly represented prior to any further analysis.

The ability to read in, clean, and understand data sets is a crucial skill. Data do not typically come in a convenient form. Careful consideration should be given to exploring the data to ensure full understanding of the nature of the data set.

Example Data Set

First, we will read in a data set that we will use in our following examples.

Visit the website

https://www.sec.gov/opa/data/market-structure/marketstructuredownloadshtml-by_security.html

and download the data from the first quarter of 2017. Create a directory for working on these examples, and place this file in that directory.

These data come from the U.S. Securities and Exchange Commission repository on market structure. This data set contains daily characteristics of a range of market variables over 5000 equities and ETFs, accumulated over several exchanges.

Be sure that the working directory is correctly set in R.

Now use the `read.table()` function to read in the file:

```
> fulldata = read.table("q1_2017_all.csv", sep=",",
+                       header=T,quote="")
```

This is a "csv" file, i.e., there are "comma-separated-values". This leads to the use of `sep=","`. The `header=T` argument specifies that the first line of the file gives the variable names.

Exercise: What happens if you use `read.table()` in the example above without `quote=""`?

Understanding the Variables

Whenever a new data set is encountered, one should fully understand its variables. Specific questions to address for each variable:

1. Describe the information that is stored in this column.
2. What type of variable is it? (Timestamp, ordinal factor, categorical factor, ratio scale, other?) Be sure that R represents the variable correctly.
3. Are there any missing values? What is the source of the missingness?
4. Are there any extreme/inappropriate values? What is the source of the extreme value?

Look at the variables:

```
> names(fulldata)
 [1] "Date"                        "Security"
 [3] "Ticker"                      "McapRank"
 [5] "TurnRank"                    "VolatilityRank"
 [7] "PriceRank"                   "LitVol..000."
 [9] "OrderVol..000."              "Hidden"
[11] "TradesForHidden"             "HiddenVol..000."
[13] "TradeVolForHidden..000."     "Cancels"
[15] "LitTrades"                   "OddLots"
[17] "TradesForOddLots"            "OddLotVol..000."
[19] "TradeVolForOddLots..000."
```

Background on the data can be found in the associated `README` file, available on Canvas in the "Data Sets" folder in the "Files" page. We will reference portions of it below.

Column 1: Date

The first column in `fulldata` is a date stamp for the observation. R has special functions for handling dates which will prove useful later when working with time series.

The `as.Date()` function transforms into the date format:

```
> print(fulldata$Date[1])   #  This is Jan. 3, 2017
[1] 20170103
```

```
> fulldata$Date = as.Date(as.character(fulldata$Date),
+                         format="%Y%m%d")
```

Note that it is necessary to cast the dates as strings, and then specify the format of those strings.

Columns 2 and 3: Security and Ticker

The variables `Security` and `Ticker` are already appropriately treated as factors by R:

```
> is.factor(fulldata$Sec) & is.factor(fulldata$Tick)
[1] TRUE
```

We also note that `Security` is either `Stock` or `ETF` (Exchange Traded Fund), and that there are over 5,000 equities under consideration:

```
> levels(fulldata$Security)
[1] "ETF"   "Stock"
```

```
> nlevels(fulldata$Ticker)
[1] 5338
```

Exercise: How many of the different ticker symbols are ETFs?

Exercise: Execute the commands below, and discuss what you find:

```
> length(unique(fulldata$Date))
> table(table(fulldata$Ticker))
> which.max(table(fulldata$Ticker))
> fulldata[duplicated(fulldata[,c(1,3)]),]
```

We will remove the duplicated lines using the following command:

```
> fulldata = fulldata[!duplicated(fulldata[,c(1,3)],
+                                 fromLast=TRUE),]
```

Note that by using `fromLast=TRUE`, we remove the **first** of each pair of duplicates.

Columns 4 through 7: Rank Variables

The next four columns provide ranks of the stocks/ETFs with respect to key attributes of Market Capitalization, Turnover, Volatility, and Price.

In general, we would prefer to start with data in its most "raw" format, and one could construct variables such as these from that data. Alas, we are often left to work with what is avaiable.

We will change these into ordered factors to reflect their ordinal nature:

```
> fulldata$McapRank =
+   factor(fulldata$McapRank, ordered=TRUE)
> fulldata$TurnRank =
+   factor(fulldata$TurnRank, ordered=TRUE)
> fulldata$VolatilityRank =
+   factor(fulldata$VolatilityRank, ordered=TRUE)
> fulldata$PriceRank =
+   factor(fulldata$PriceRank, ordered=TRUE)
```


Columns 8 thru 19: The Count Variables

The remaining column in the data set consist of trade and share counts of different types.

Note that some of the volume counts are reported in 1000's of shares, hence there are non-integer values among the counts.

The `summary()` function is a useful first step in understanding the basic properties of the variables. A primary concern is the presence of extreme or impossible values.

```
> summary(fulldata[,8:19])
 LitVol..000.       OrderVol..000.        Hidden        TradesForHidden
Min.   :     0.00   Min.   :      0   Min.   : -143.0   Min.   :     0
1st Qu.:     3.46   1st Qu.:    885   1st Qu.:   10.0   1st Qu.:    41
Median :    44.57   Median :   4003   Median :   75.0   Median :   501
Mean   :   431.76   Mean   :  35747   Mean   :  454.4   Mean   :  3476
3rd Qu.:   254.41   3rd Qu.:  15581   3rd Qu.:  370.0   3rd Qu.:  2846
Max.   :119014.06   Max.   :7293901   Max.   :89335.0   Max.   :533874
HiddenVol..000.    TradeVolForHidden..000.    Cancels
Min.   :-7903.64   Min.   :     0.00        Min.   :      0
1st Qu.:    1.15   1st Qu.:     5.64        1st Qu.:   3126
Median :    8.74   Median :    55.31        Median :  15583
Mean   :   66.91   Mean   :   498.04        Mean   :  73579
3rd Qu.:   44.02   3rd Qu.:   301.58        3rd Qu.:  60112
Max.   :14821.94   Max.   :133714.59        Max.   :7607714
```

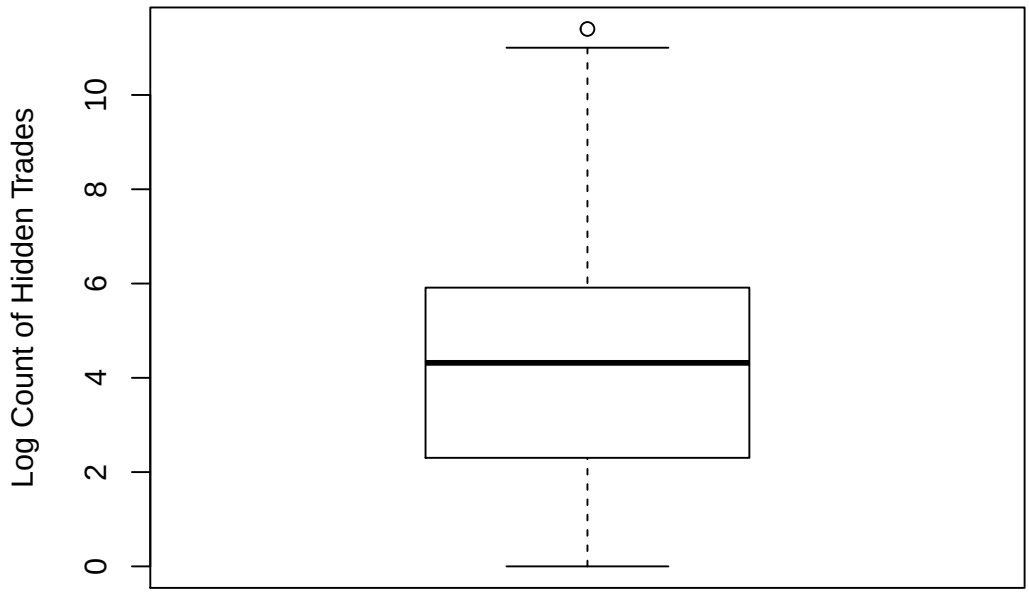


```
   LitTrades          OddLots        TradesForOddLots OddLotVol..000.
Min.   :     0   Min.   :     0   Min.   :     0   Min.   :   0.000
1st Qu.:    23   1st Qu.:     8   1st Qu.:    39   1st Qu.:   0.280
Median :   377   Median :   154   Median :   464   Median :   5.254
Mean   :  2708   Mean   :   916   Mean   :  3115   Mean   :  32.492
3rd Qu.:  2186   3rd Qu.:   887   3rd Qu.:  2558   3rd Qu.:  31.083
Max.   :383222   Max.   :130106   Max.   :462414   Max.   :3728.885
TradeVolForOddLots..000.
Min.   :     0.00
1st Qu.:     5.31
Median :    49.94
Mean   :   422.78
3rd Qu.:   260.18
Max.   :108034.79
```

Of course, graphical tools are useful for understanding the properties of variables. This topic will be covered more fully in the next Part, but here we consider a classic approach: the boxplot:

```
> boxplot(log(fulldata$Hidden),
+         ylab="Log Count of Hidden Trades")
```

The "box" of the plot extends from the 25th to the 75th percentile of the data, while the solid line in the middle of the box is at the median. The two "arms" extend to the minimum and maximum value, **except** that any value labelled an outlier is plotted separately.



Missing Data

Most data sets contain some amount of missing data, for many reasons.

R uses the special symbol `NA` to represent a missing value.

As a first step, it is important to recognize how your data set represents missing values, so that R handles them properly. The function `read.table()` has an argument `na.strings` to allow one to specify missing values. **Be careful:** It is not unusual for data sets to use numbers such as `-999` to indicate a missing value.

In a `.csv` file such as that we are using here, it is common to simply have blank values to indiciate missingness, i.e., there are consecutive commas without a value between.

Exercise: Does R appropriately handle cases where a missing value is indicated using blank values in a `.csv` file?

A useful tool is the function `md.pattern()` that is part of package `mice`. This function shows the different "patterns" of missingness in the data set, and how often they occur.

Consider the example on the following page, the result of calling

```
> md.pattern(data.frame(fulldata), plot=FALSE)
```

This shows that there are 325,082 complete cases in the data set. There is one case where `VolatilityRank` is missing, and there are 76 cases where `McapRank`, `TurnRank` and `PriceRank` are missing.


```
        Date Security Ticker LitVol..000. OrderVol..000. Hidden
325082    1        1      1            1              1      1
76        1        1      1            1              1      1
1         1        1      1            1              1      1
          0        0      0            0              0      0
        TradesForHidden HiddenVol..000. TradeVolForHidden..000. Cancels
325082                1               1                       1       1
76                    1               1                       1       1
1                     1               1                       1       1
                      0               0                       0       0
        LitTrades OddLots TradesForOddLots OddLotVol..000.
325082         1       1                1               1
76             1       1                1               1
1              1       1                1               1
               0       0                0               0
        TradeVolForOddLots..000. VolatilityRank McapRank TurnRank PriceRank
325082                        1              1        1        1         1
76                            1              1        0        0         0
1                             1              0        1        1         1
                              0              1       76       76        76

325082   0
76       3
1        1
        229
```

Exercise: Inspect the output from

```
> fulldata[!complete.cases(fulldata),]
```

What does this say about the cases with missing values?

Of course, more sophisticated visualization of the cases with missing data would likely provide more information as to the source of the missingness. In the next Part we will consider such tools.

Part 5: Visualization

Summary

This Part covers tools for visualizing data in R.

Powerful visualization techniques are critical to discovering and understanding the relationships between variables.

Example Data Set

First, we will revisit the data set we considered in the previous Part, and the steps needed to properly format the data:

```
> fulldata = read.table("q1_2017_all.csv", sep=",", header=T, quote="")
> fulldata$Date = as.Date(as.character(fulldata$Date), format="%Y%m%d")
> fulldata = fulldata[!duplicated(fulldata[,c(1,3)], fromLast=TRUE),]
> fulldata$McapRank = factor(fulldata$McapRank, ordered=TRUE)
> fulldata$TurnRank = factor(fulldata$TurnRank, ordered=TRUE)
> fulldata$VolatilityRank = factor(fulldata$VolatilityRank, ordered=TRUE)
> fulldata$PriceRank = factor(fulldata$PriceRank, ordered=TRUE)
```

The entire data set is a bit large to work with conveniently, so we are going to restrict consideration to 1000 equities:

```
> tickkeep = unique(fulldata$Ticker)[seq(1,
+       length(unique(fulldata$Ticker)),length=100)]
> fulldata = fulldata[fulldata$Ticker %in% tickkeep,]
```

Classic Plotting Functions in R

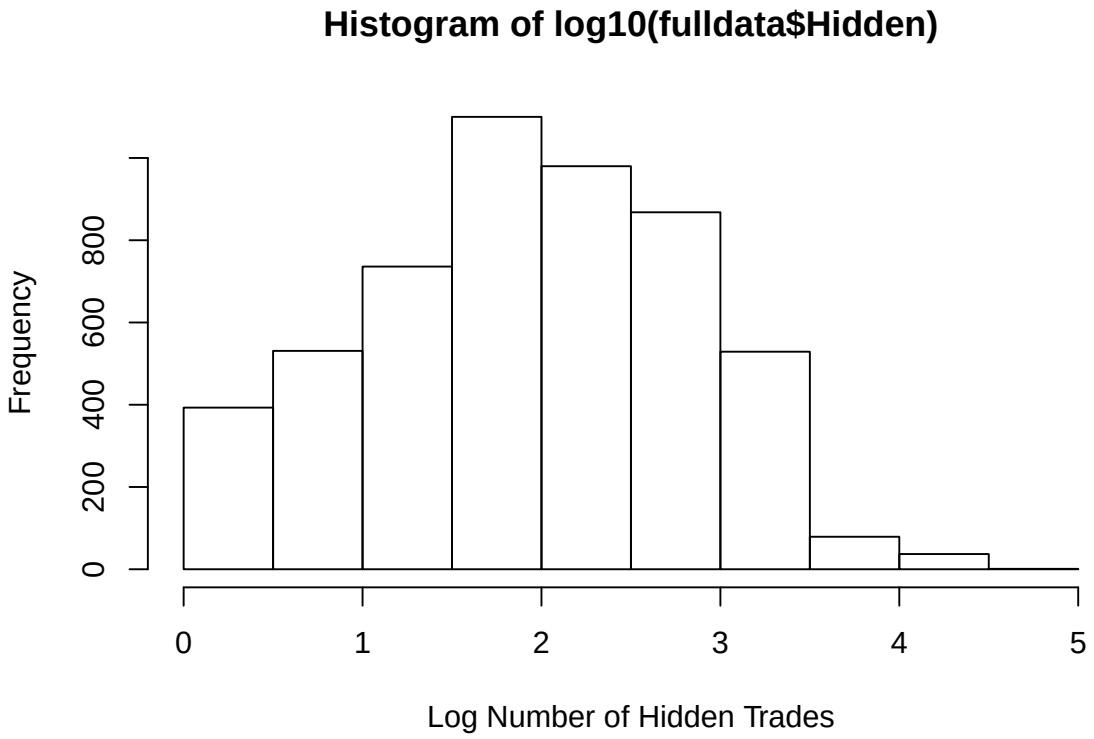
Prior to moving on the more advanced visualization functions, it is important to understand the basic plotting capabilities of R.

We saw the use of `boxplot()` in a previous Part. A similar view of the shape of a distribution can be obtained using a histogram, formed using the function `hist()`:

```
> hist(log10(fulldata$Hidden),
+      xlab="Log Number of Hidden Trades")
```

The title on the plot can be changed using the `main` argument.

The number of bins in the histogram can be altered using the `breaks` argument.

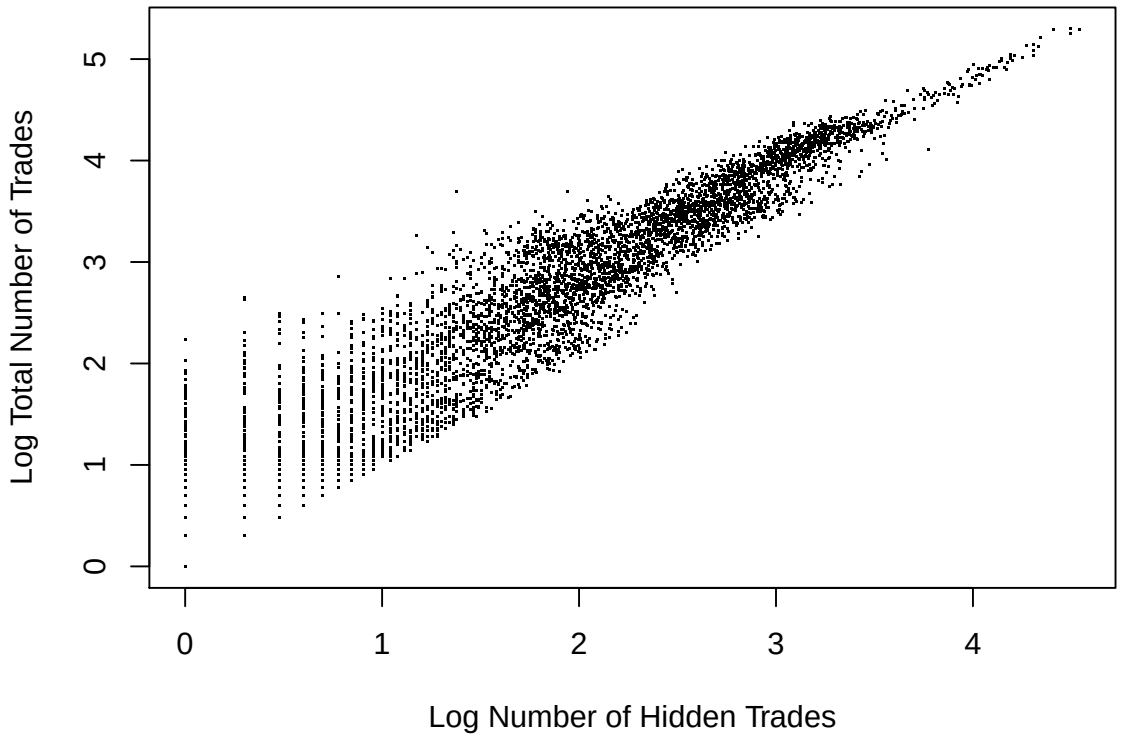


Scatter plots are ubiquitous, as well. They are created simply in R using the `plot()` function:

```
> plot(log10(fulldata$Hidden),
+      log10(fulldata$TradesForHidden),
+      xlab="Log Number of Hidden Trades",
+      ylab="Log Total Number of Trades", pch=".")
```

The argument `pch` changes the plotting character. In a case such as this, with so many dots on the plot, the default plotting character is too large.

Many other properties of the scatter plot can be adjusted using the `par()` function. See `help(par)` for more information.



Introduction to ggplot2

The package `ggplot2` contains a great deal of valuable data visualization tools. This package can be installed, along with other useful packages for working with large and complex data sets, via the `tidyverse` meta-package:

```
> install.packages("tidyverse")
```

The collection of packages in tidyverse comprise "a collection of R packages that share common philosophies and are designed to work together." – tidyverse.org

The `ggplot2` package implements a grammar of graphics, where plots are created by combining geoms.

The syntax is a little awkward at first, but this approach yields a great deal of flexibility, and a great deal of power in exploring features in a data set.

A plot is initialized by using the `ggplot()` function:

```
> ggplot(data = fulldata)
```

To the basic plot, one adds the "aesthetic" via the `mapping` option. An aesthetic a generic concept for the visual features of a plot. In this case, we are specifying that the variable on the horizontal axis is `Hidden`, and the variable on the vertical axis is `Cancels`:

```
> ggplot(data = fulldata, mapping =
+                  aes(x=Hidden, y=Cancels))
```

We can now add onto this base to create plots with complex features. You can, for instance, define

```
> baseplot = ggplot(data = fulldata, mapping =
+                  aes(x=Hidden, y=Cancels))
```

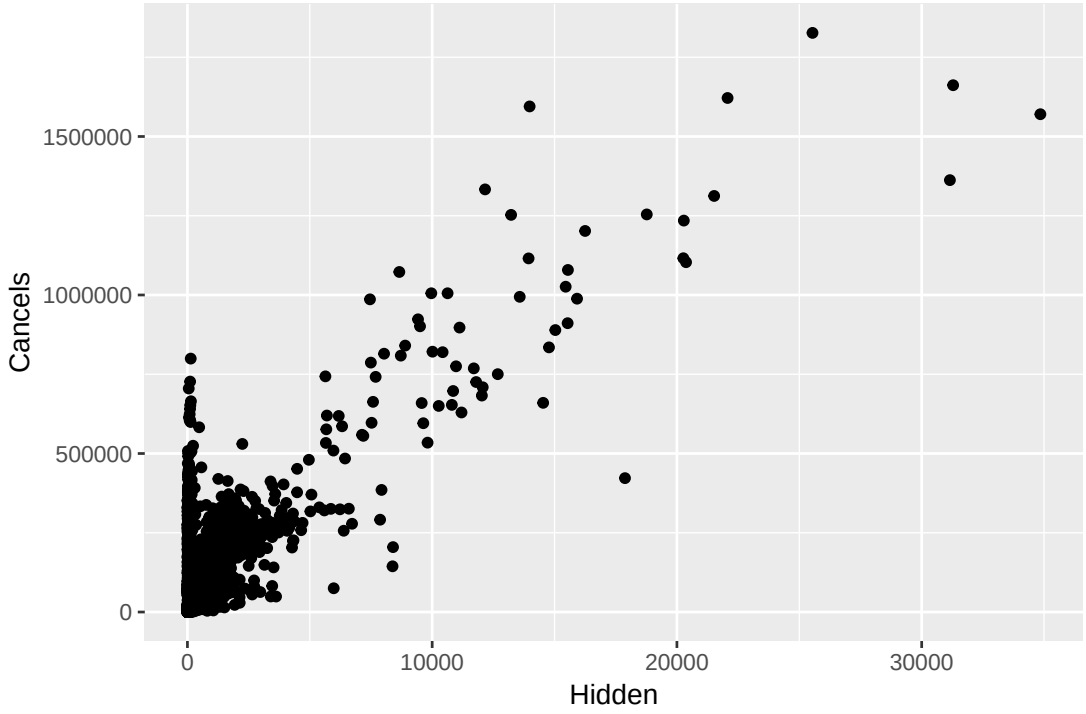
and add onto `baseplot` in what follows.

A Simple Scatter Plot

Consider the following syntax:

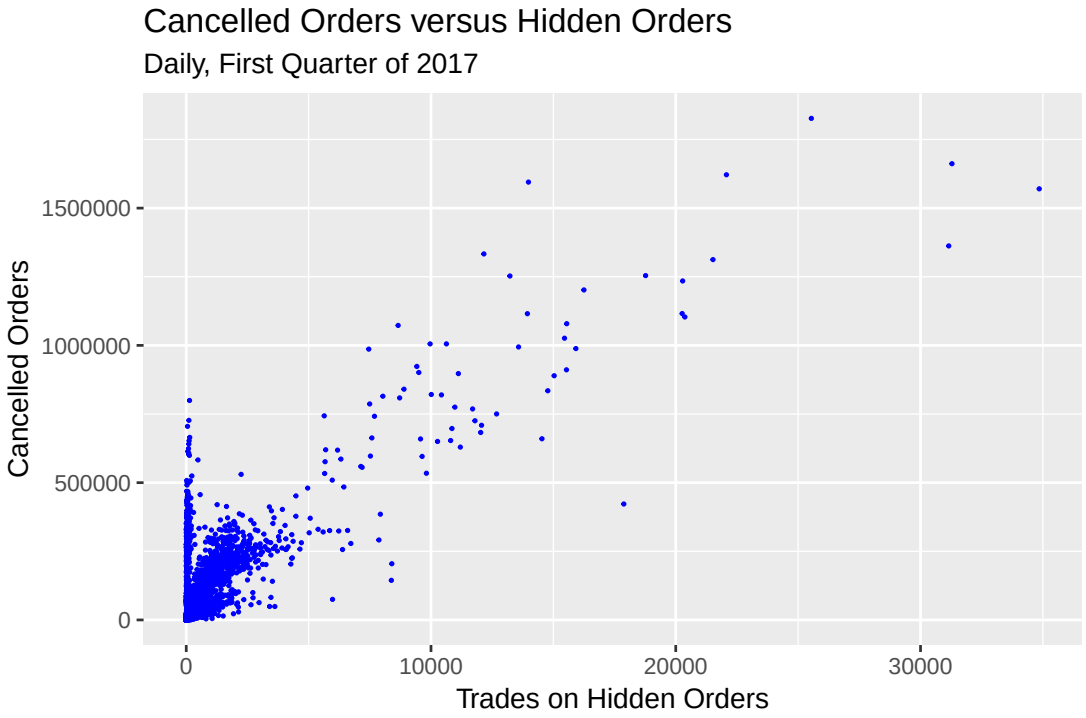
```
> baseplot + geom_point()
```

Note how the `geom_point()` function is used to add a scatter plot onto the current figure, as stored in `baseplot`.



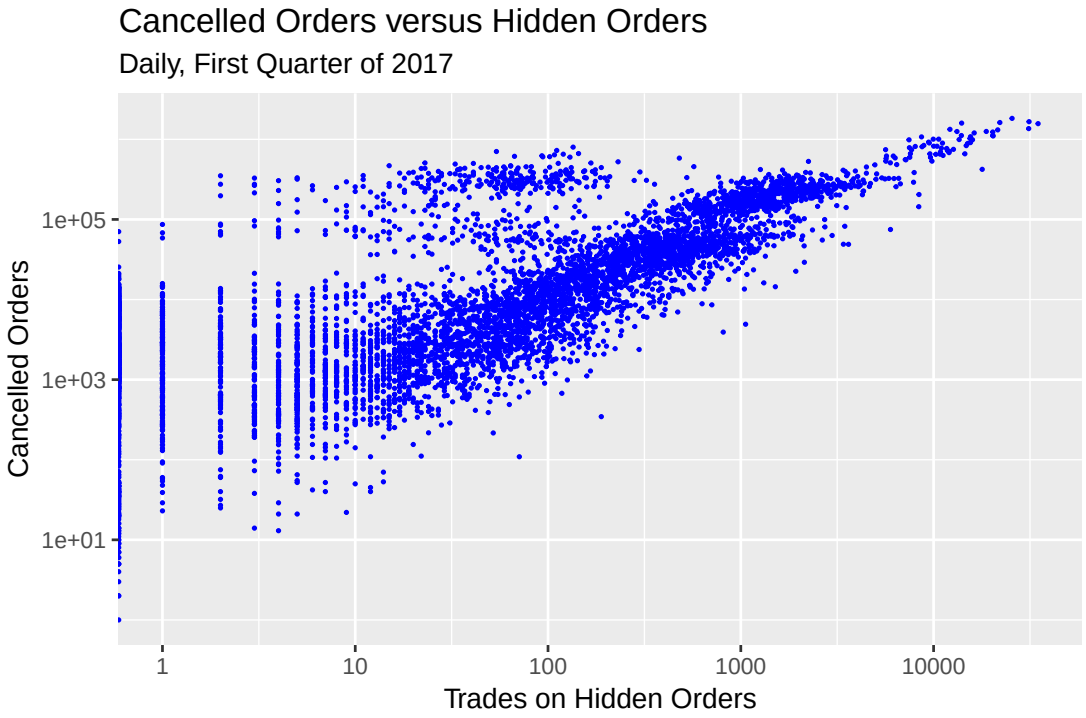
Now we will work to improve the appearance of this plot.

```
> baseplot +
+   geom_point(size=0.3, color="blue") +
+   labs(x="Trades on Hidden Orders",
+        y="Cancelled Orders",
+        title="Cancelled Orders versus Hidden Orders",
+        subtitle="Daily, First Quarter of 2017")
```



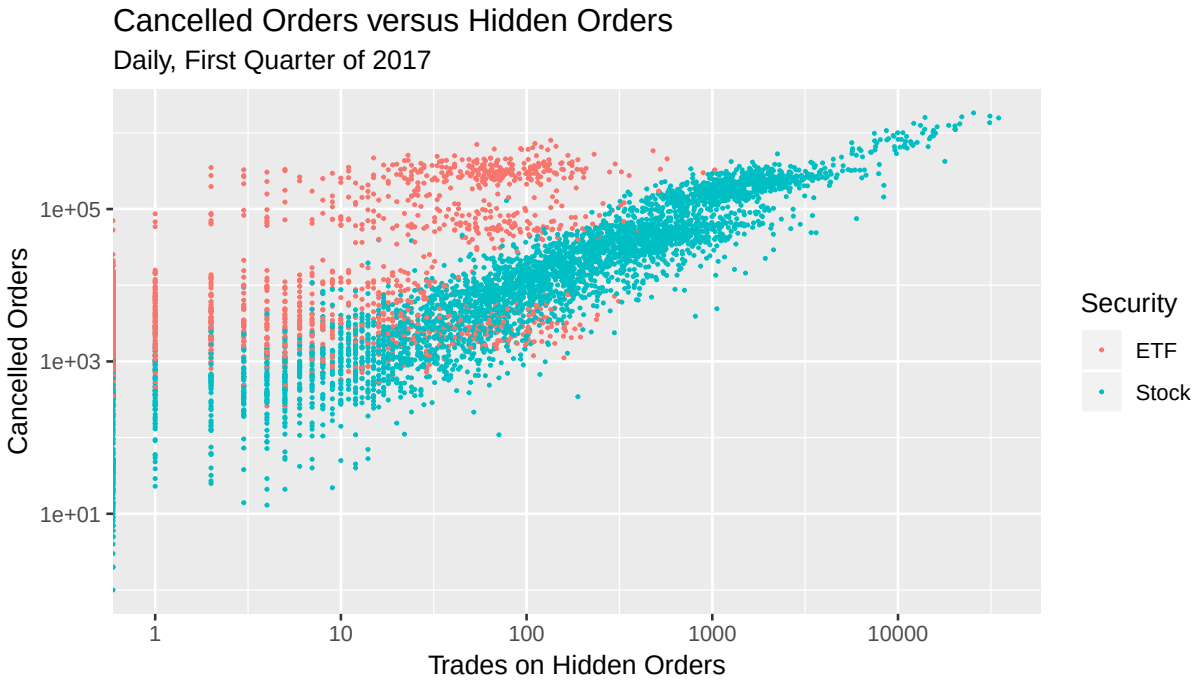
Next, we will convert the axes to the log scale:

```
> baseplot +
+   geom_point(size=0.3, color="blue") +
+   labs(x="Trades on Hidden Orders",
+      y="Cancelled Orders",
+      title="Cancelled Orders versus Hidden Orders",
+      subtitle="Daily, First Quarter of 2017") +
+   scale_x_log10() + scale_y_log10()
```



The `color` argument can be switched to vary with the levels of one of the factors.

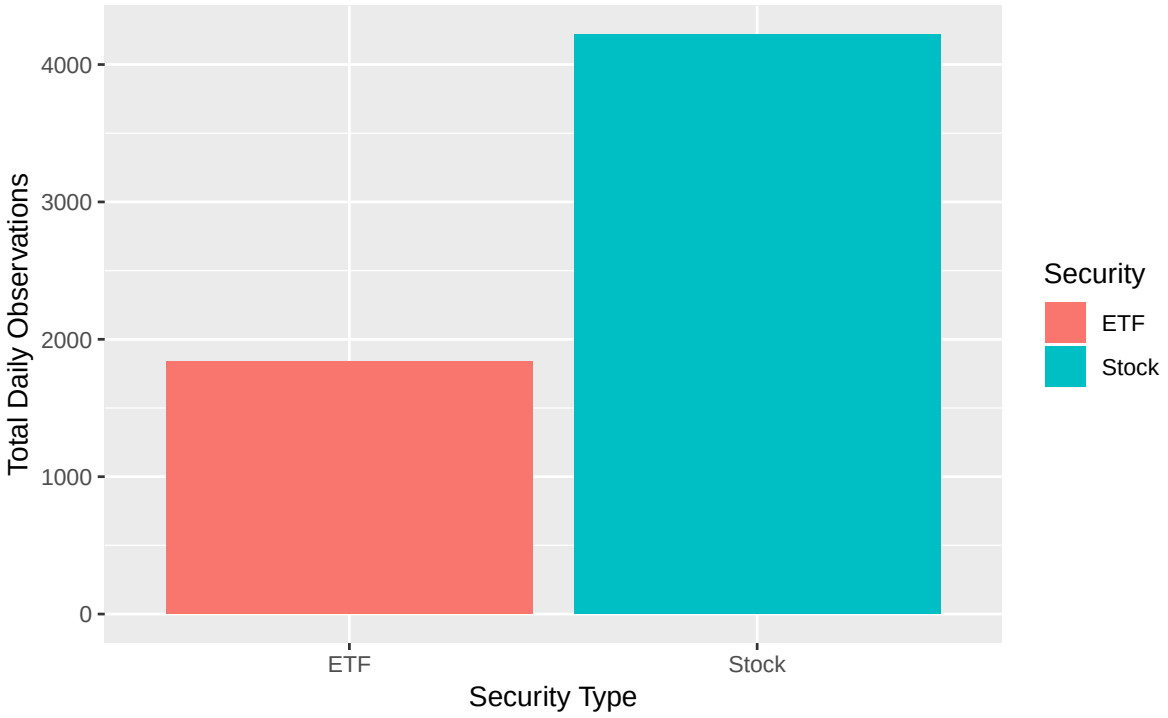
```
> baseplot +
+   geom_point(size=0.3, mapping=aes(color=Security)) +
+   labs(x="Trades on Hidden Orders",
+        y="Cancelled Orders",
+        title="Cancelled Orders versus Hidden Orders",
+        subtitle="Daily, First Quarter of 2017") +
+   scale_x_log10() + scale_y_log10()
```



Bar Plots

Basic bar charts can be created to inspect the distribution of a factor, using the `geom_bar()` function:

```
> ggplot(data=fulldata,
+     mapping=aes(x=Security, fill=Security)) +
+   geom_bar() +
+   labs(x="Security Type",
+        y="Total Daily Observations")
```

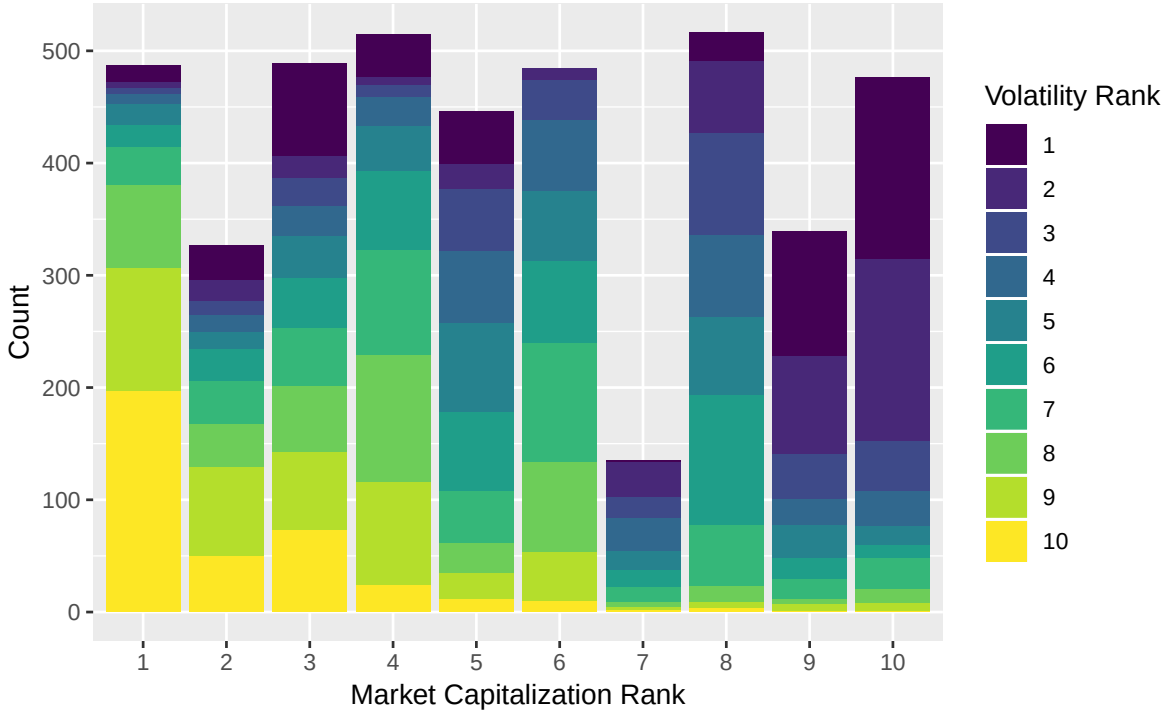


For the next plot, we will restrict to data that do not have missing values on `McapRank`, and are of type "Stock":

```
> fulldataStock = filter(fulldata,
+         Security == "Stock" & !is.na(McapRank))
```

We will adjust the aesthetic so that the "fill" color of the bars corresponds to the level of `VolatilityRank`:

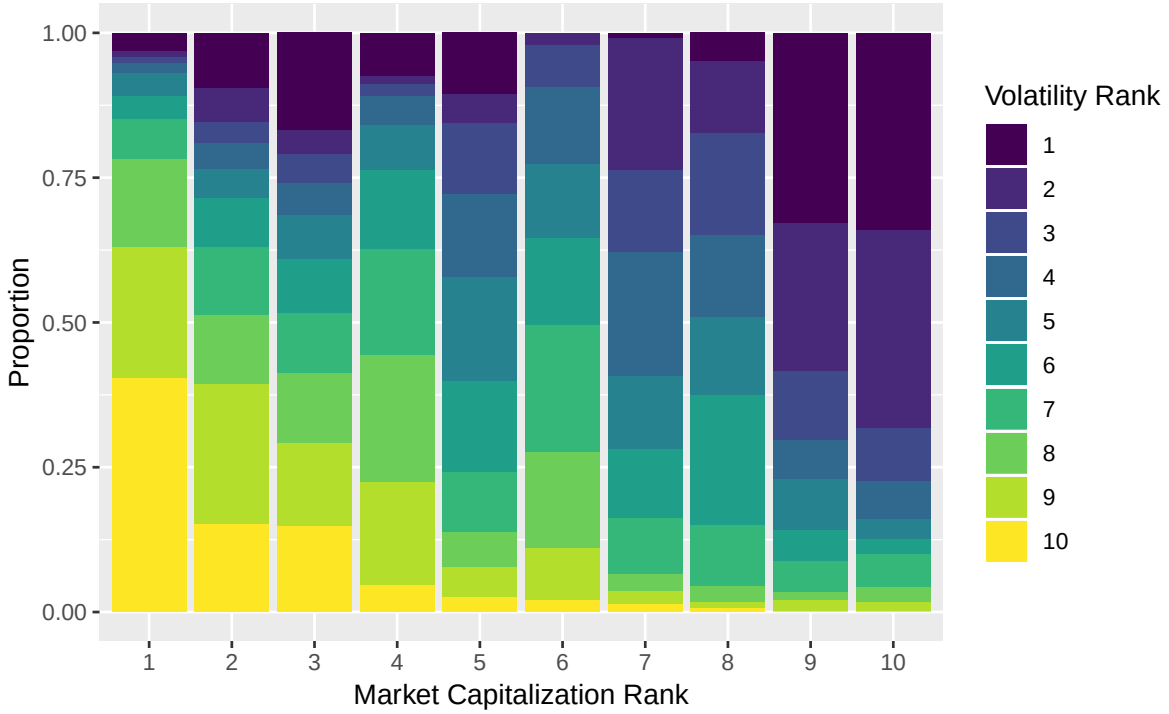
```
> ggplot(data=fulldataStock,
+     mapping=aes(x=McapRank, fill=VolatilityRank)) +
+   geom_bar() +
+   labs(x="Market Capitalization Rank", y="Count",
+     fill="Volatility Rank")
```



The bars in the previous plot are not of equal height because of the sub-sampling we did to reduce to 1000 equities.

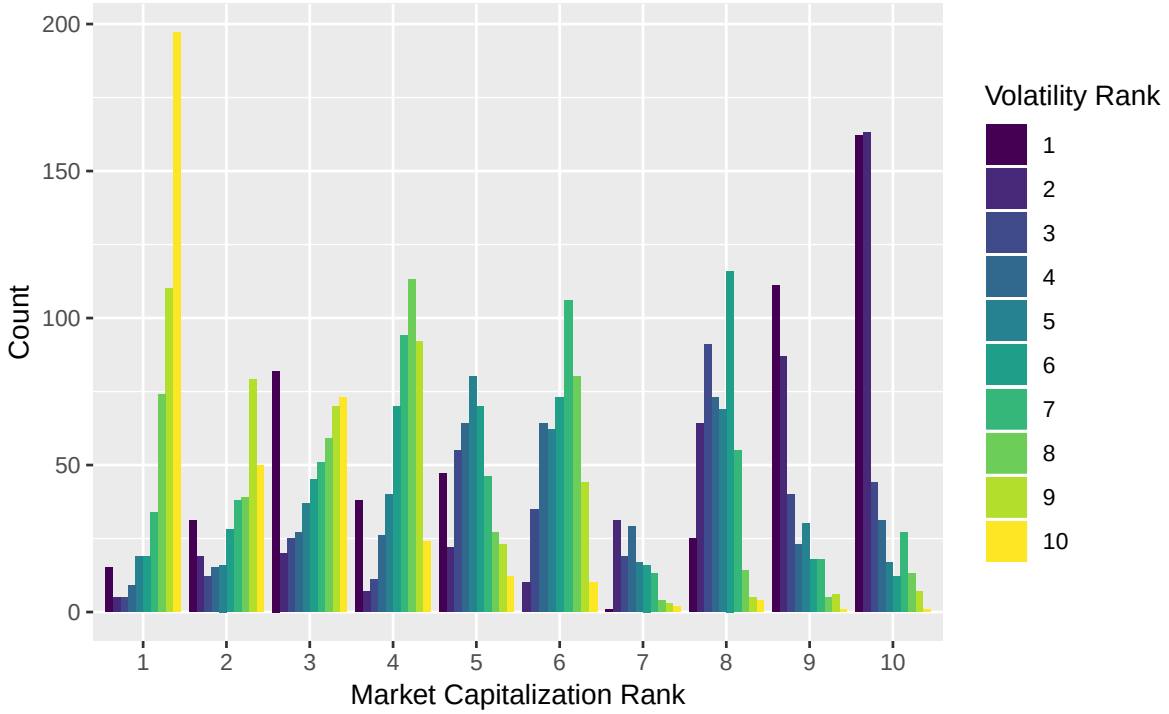
We can change vertical axis to proportion by setting `position="fill"`:

```
> ggplot(data=fulldataStock,
+     mapping=aes(x=McapRank, fill=VolatilityRank)) +
+   geom_bar(position="fill") +
+   labs(x="Market Capitalization Rank",
+     y="Proportion", fill="Volatility Rank")
```



By setting `position="dodge"`, the bars are arranged more like a histogram:

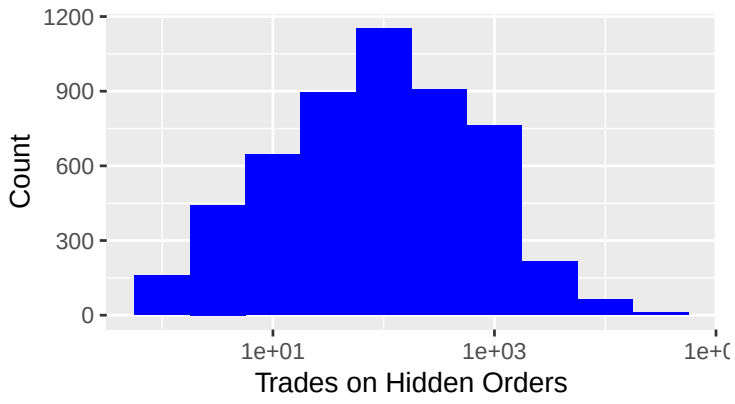
```
> ggplot(data=fulldataStock,
+     mapping=aes(x=McapRank, fill=VolatilityRank)) +
+   geom_bar(position="dodge") +
+   labs(x="Market Capitalization Rank",
+     y="Count", fill="Volatility Rank")
```



Histograms

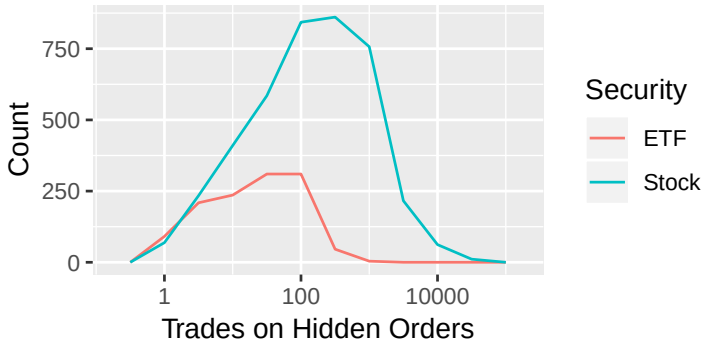
The function `geom_histogram()` will create a histogram:

```
> ggplot(data=fulldata, mapping=aes(x=Hidden)) +
+   geom_histogram(binwidth=0.5, fill="blue") +
+   scale_x_log10() +
+   labs(x="Trades on Hidden Orders", y="Count")
```



Overlapping histograms can be difficult to interpret (try it!). An alternative is to create frequency polygons using `geom_freqpoly()`:

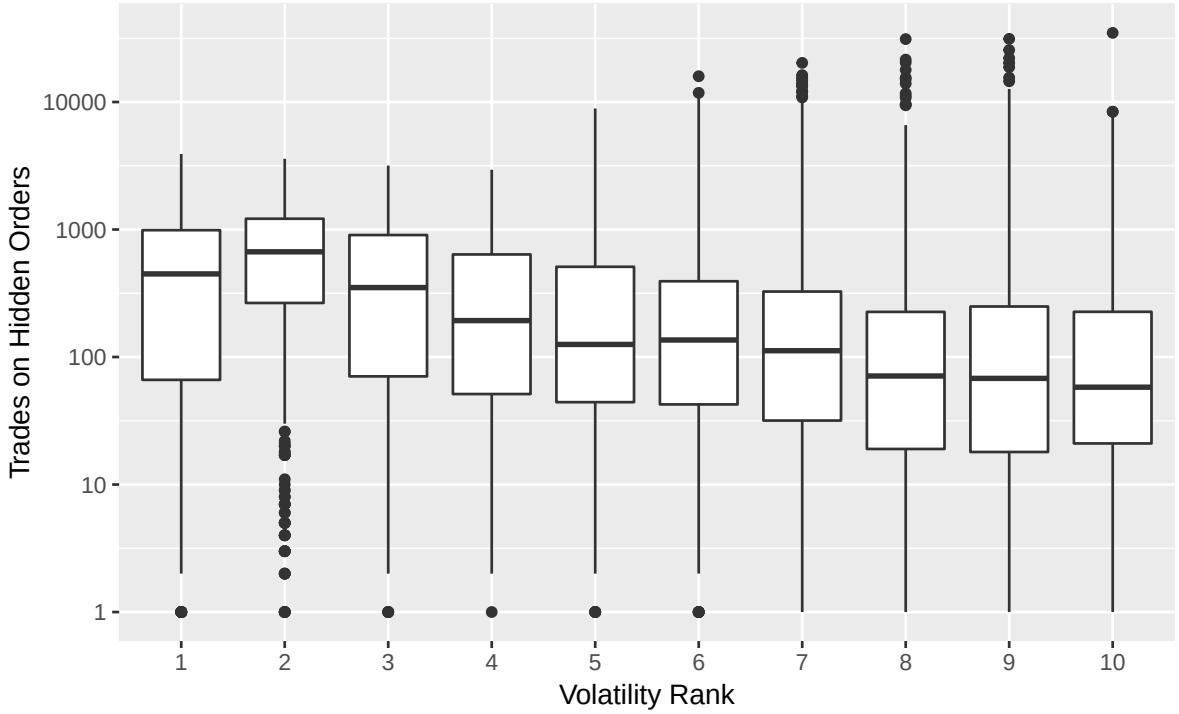
```
> ggplot(data=fulldata,
+        mapping=aes(x=Hidden, color=Security)) +
+   geom_freqpoly(binwidth=0.5, fill="blue") +
+   scale_x_log10() +
+   labs(x="Trades on Hidden Orders", y="Count")
```



Boxplots

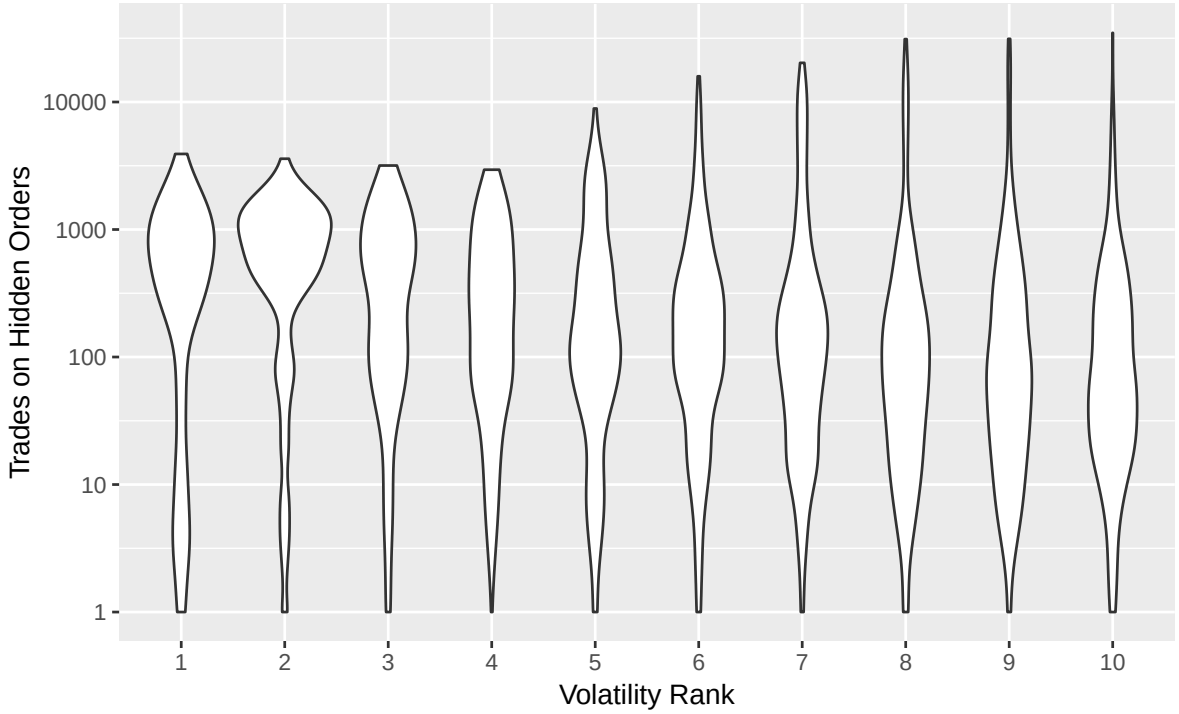
Side-by-side boxplots are also useful for comparing the distributions of variables over different values of a factor. For example,

```
> ggplot(data=fulldataStock,
+        mapping=aes(x=VolatilityRank, y=Hidden)) +
+   geom_boxplot() + scale_y_log10() +
+   labs(y="Trades on Hidden Orders",
+        x="Volatility Rank")
```



A violin plot shows the same type of information, with a little more detail. The width of a "violins" is greater in areas where a greater proportion of the observations lie.

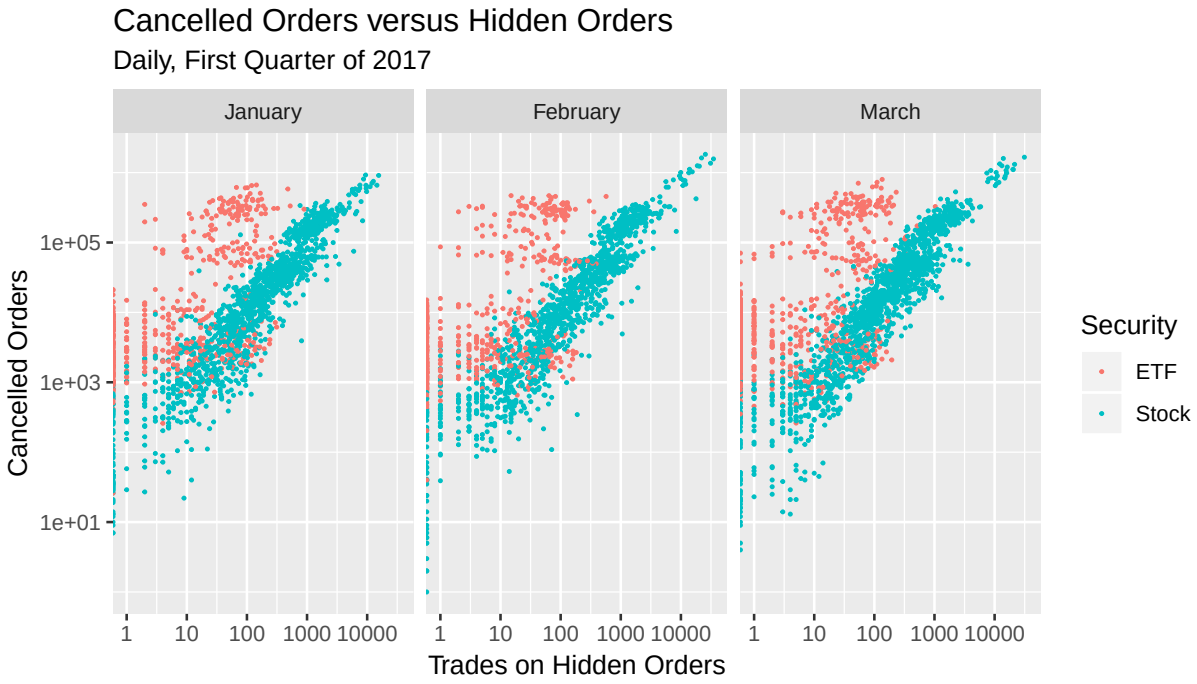
```
> ggplot(data=fulldataStock,
+        mapping=aes(x=VolatilityRank, y=Hidden)) +
+   geom_violin() + scale_y_log10() +
+   labs(y="Trades on Hidden Orders",
+        x="Volatility Rank")
```



Facets

Facets refer to an arrangment of plots on which one or more factors vary. For example, return to our scatter plot example above, and use the `facet_grid()` function to break up the plots by month:

```
> baseplot +
+   geom_point(size=0.3, mapping=aes(color=Security)) +
+   labs(x="Trades on Hidden Orders",
+        y="Cancelled Orders",
+        title="Cancelled Orders versus Hidden Orders",
+        subtitle="Daily, First Quarter of 2017") +
+   scale_x_log10() + scale_y_log10() +
+   facet_grid(.~factor(months(Date),
+         levels=c("January","February","March")))
```



Exercise: What happens in the previous example if the command

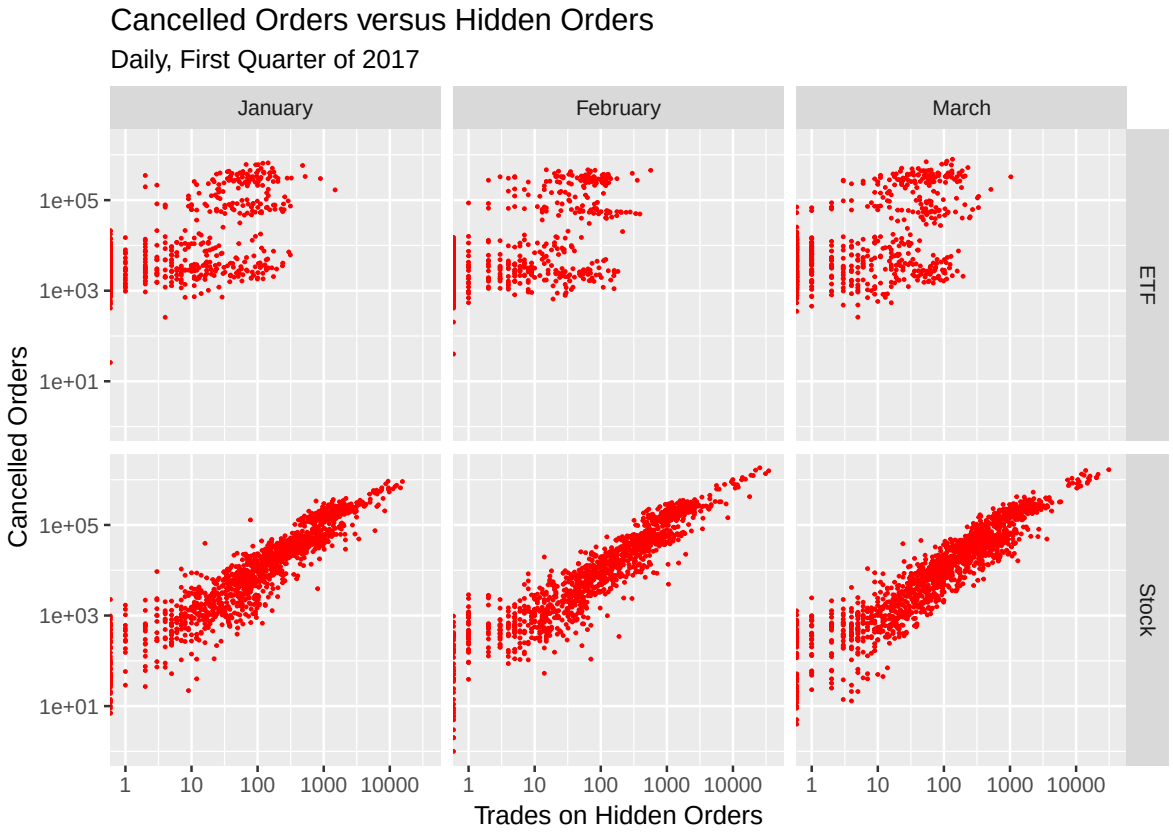
```
> facet_grid(.~months(Date))
```

is used instead? What about if use the following?

```
> facet_grid(months(Date)~.)
```

Two factors can vary:

```
> baseplot +
+   geom_point(size=0.3, color="red") +
+   labs(x="Trades on Hidden Orders",
+        y="Cancelled Orders",
+        title="Cancelled Orders versus Hidden Orders",
+        subtitle="Daily, First Quarter of 2017") +
+   scale_x_log10() + scale_y_log10() +
+   facet_grid(Security~factor(months(Date),
+         levels=c("January","February","March")))
```

Time Series Plots

We will, of course, be working with a great deal of time series data. The `geom_line()` function is useful:

```
> ggplot(data=filter(fulldata,Ticker=="AMD"),
+        mapping=aes(x=Date,y=Hidden)) +
+   geom_line(color="blue") +
+   labs(x="Date", y="Trades on Hidden Orders",
+      title="Change in Hidden Orders over Time",
+      subtitle="AMD, daily, first quarter of 2017")
```